

Exercicio 1: Despregue de aplicación Laravel

Explicación O primeiro paso sempre debería ser a posta en produción do *backend* xunto a base de datos de produción. Comezaremos por **crear a base de datos**. Utilizaremos o **plan gratuito de Aiven**. O plan gratuito está dispoñible para os servizos de PostgreSQL, MySQL e Valkey (un base de datos tipo Redis).

1. Crea unha conta co GMail do IES en <https://console.aiven.io/signup>

2. Elixe `Personal Project`

, `Experiment with new technologies` e `Developer`

.

Vai a `Create new service`

1. . E selecciona unha base de datos PostgreSQL. O nome do servizo da igual, selecciona o plan gratuito e despois un servidor que estea en Europa. Espera a que teñas preparado o servizo. Unha vez que a teñas darache os datos de conexión a base de datos. Garda estes datos de algún xeito. Senón, sempre podes volver a recuperalos entrando no servido de Aiven.

Explicación A continuación **clonaremos a nosa aplicación de *backend*** do seguinte [repositorio](#). Esta conta con 2 versións etiquetadas con `tags`, polo que comezaremos poñendo en marcha a **versión 1**.

1. Inicia a máquina virtual `dapw_iniciais_server_01`

e conéctate por SSH dende o Visual Studio Code.

Clona o repositorio anterior en `/home/dadmin/`

.

Sitúate no `commit`

1. coa a etiqueta `v1.0`. Para iso executa o seguinte comando:

```
$ git checkout v1.0
```

Explicación Agora xa temos o código fonte da aplicación que imos poñer en produción no servidor. Pero esta conta con **dependencias** que aínda non foron instaladas. No caso de Laravel, **poderémolas instalar mediante** `Composer`

.

1. Instala `Composer`

.

Sitúate no directorio `src` que onde se atopa o código da aplicación.

Instala as dependencias con `Composer`

1. . Recorda non instalar aquelas que son para desenvolvemento. **Entrega captura** do comando utilizado.

```

o dadmin@cvdserver01:~/api-rest-en-laravel/src$ composer install --no-dev
Installing dependencies from lock file
Verifying lock file contents can be installed on current platform.
Package operations: 79 installs, 0 updates, 0 removals
  - Downloading doctrine/inflector (2.1.0)
  - Downloading doctrine/lexer (3.0.1)
  - Downloading symfony/polyfill-ctype (v1.33.0)
  - Downloading webmozart/assert (1.11.0)
  - Downloading dragonmantank/cron-expression (v3.4.0)
  - Downloading symfony/polyfill-php83 (v1.33.0)
  - Downloading symfony/polyfill-mbstring (v1.33.0)
  - Downloading symfony/deprecation-contracts (v3.6.0)
  - Downloading symfony/http-foundation (v7.3.3)
  - Downloading fruitcake/php-cors (v1.3.0)
  - Downloading psr/http-message (2.0)
  - Downloading psr/http-client (1.0.3)
  - Downloading ralouphie/getallheaders (3.0.3)
  - Downloading psr/http-factory (1.1.0)
  - Downloading guzzlehttp/psr7 (2.8.0)
  - Downloading guzzlehttp/promises (2.3.0)
  - Downloading guzzlehttp/guzzle (7.10.0)
  - Downloading symfony/polyfill-php80 (v1.33.0)
  - Downloading guzzlehttp/uri-template (v1.0.5)
  - Downloading symfony/polyfill-intl-normalizer (v1.33.0)
  - Downloading symfony/polyfill-intl-grapheme (v1.33.0)
  - Downloading symfony/string (v7.3.3)
  - Downloading psr/container (2.0.2)
  - Downloading symfony/service-contracts (v3.6.0)
  - Downloading symfony/console (v7.3.3)
  - Downloading laravel/prompts (v0.3.6)
  - Downloading voku/portable-ascii (2.0.3)
  - Downloading phpoption/phpooption (1.9.4)
  - Downloading graham-campbell/result-type (v1.1.3)
  - Downloading vlucas/phpdotenv (v5.6.2)

```

Explicación A continuación modificaremos o ficheiro `.env`. No noso caso teremos que modificar dúas partes. Unha para **indicarlle** a Laravel que a aplicación **está en produción**. E outra para **configurar a conexión coa base de datos** de produción. Ademais será necesario **crear unha clave para a aplicación**.

En Laravel, `APP_ENV`

e `APP_DEBUG`

son dúas variables de configuración fundamentais que controlan o comportamento da aplicación dependendo de onde se execute. Imos explicalo con detalle:

`APP_ENV`

indica en **que entorno está correndo a aplicación**. Valores típicos:

- `local`: Desenvolvemento local.
- `production`

: Servidor en produción.

`staging`

- : Preproducción.

Explicación Laravel pode cambiar configuracións automáticas segundo o entorno. Por exemplo, os **logs**, **cache**, ou certos servizos externos poden comportarse distinto en `local` e `production`

APP_DEBUG

controla se Laravel **mostra detalles completos dos erros** ou non. Valores:

- `true`: Mostrar `stack trace`
- `true`, variables, e detalles completos do erro. Útil para desenvolvemento.
- `false`: Mostrar unha **páxina de erro xeral** (500, 404) sen filtrar información sensible. **Obrigatorio en produción.**

1. Modifícalle o nome ao ficheiro `.env.example`

por `.env`.

Modifica a liña do ficheiro `.env` `APP_DEBUG=true`

por `APP_DEBUG=false` e `APP_ENV=local` por `APP_ENV=production`

1..

APP_KEY

é unha **clave secreta de cifrado** que Laravel utiliza para:

- **Cifrar e descifrar datos** sensibles (por exemplo, cookies e sesións cifradas).
- **Protexer tokens e datos internos** da aplicación.
- Garantir que a información cifrada non poida ser descifrada por terceiros.

Explicación Sen esta clave, Laravel **non pode usar o sistema de cifrado**

Non cambies `APP_KEY`

nun proxecto en produción se xa tes datos cifrados (sesións, *cookies*), porque eses datos deixarán de ser válidos.

1. Xera unha clave `APP_KEY`

co comando `php artisan key:generate`

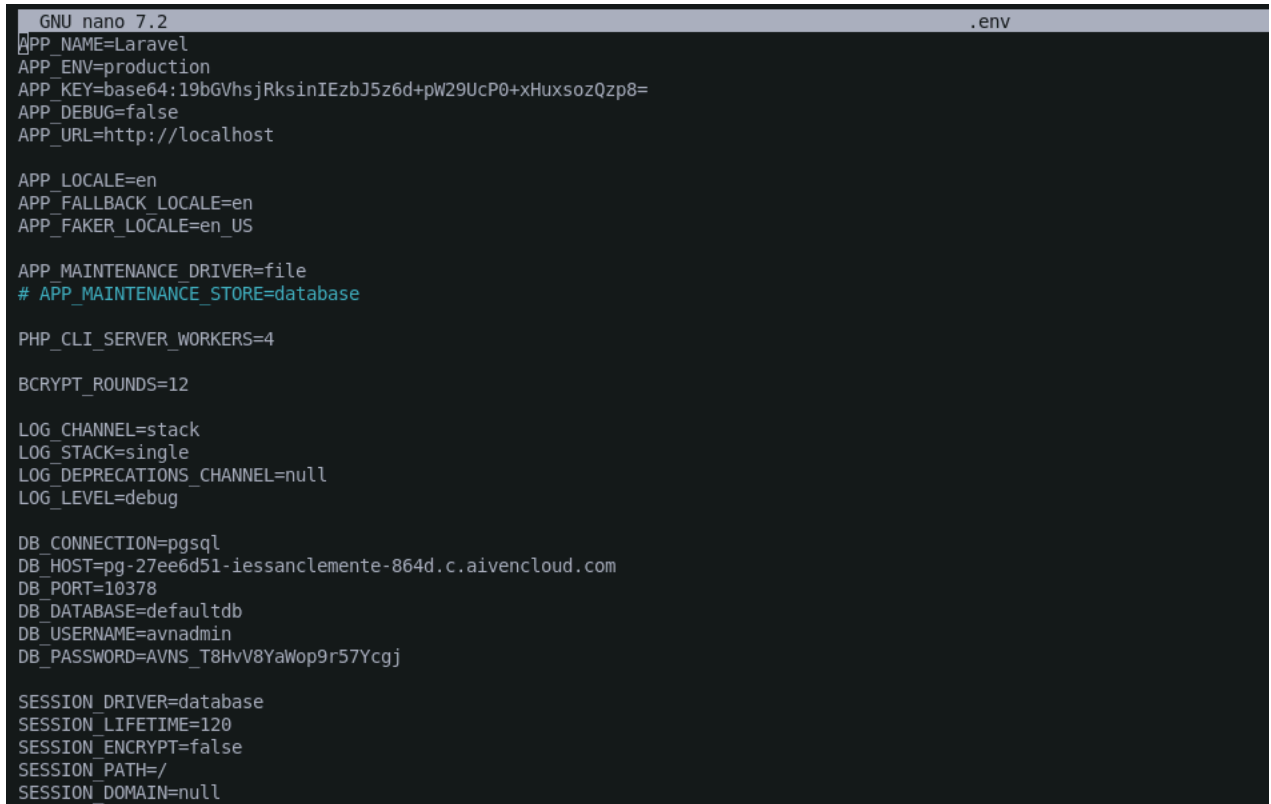
1..

Explicación A continuación faremos a **migración da base de datos**. Para poder realizala primeiro deberemos modificar o ficheiro `.env` para indicar a conexión a base de datos en produción, para que Laravel poida saber onde executar a migración.

1. Como a base de datos que vamos a utilizar é PostgreSQL, seguramente non instalamos a extensión de PHP para poder conectarse a este tipo de base de datos. Polo tanto instala o driver e reinicia Apache para que este poida cargar dita extensión.

```
sudo apt update
sudo apt install php-pgsql
sudo service apache2 restart
```

1. Modifica os datos da conexión a base de datos polos obtidos nos pasos anteriores no ficheiro `.env`. O tipo de base de datos é `pgsql`. **Entrega capturas** do ficheiro `.env`.



```
GNU nano 7.2 .env
APP_NAME=Laravel
APP_ENV=production
APP_KEY=base64:19bGVhsjRksinIEzbJ5z6d+pW29UcP0+xHuxsozQzp8=
APP_DEBUG=false
APP_URL=http://localhost

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

DB_CONNECTION=pgsql
DB_HOST=pg-27ee6d51-iessanclemente-864d.c.aivencloud.com
DB_PORT=10378
DB_DATABASE=defaultdb
DB_USERNAME=avnadmin
DB_PASSWORD=AVNS_T8HvV8YaWop9r57Ycgj

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null
```

Esta é a variable co valor por defecto onde se indica o tipo de base de datos:

```
DB_CONNECTION=sqlite
...
```

1. Executa o comando `php artisan migrate`

1. para realizar a migración da base de datos. Isto creará e modificarás as táboas do noso proxecto.

Explicación Procederemos agora a consultar as táboas das base de datos creadas. Para iso utilizaremos a extensión `SQLTools`

de VSC que xa utilizamos en tarefas anteriores como **cliente de base de datos**.

1. Vaite a extensión `SQLTools`

de VSC e preme `Add New Connection`

. Se non a tes instálala.

Seguramente non vexas un driver para PostgreSQL. Así que preme en `Get more drivers`

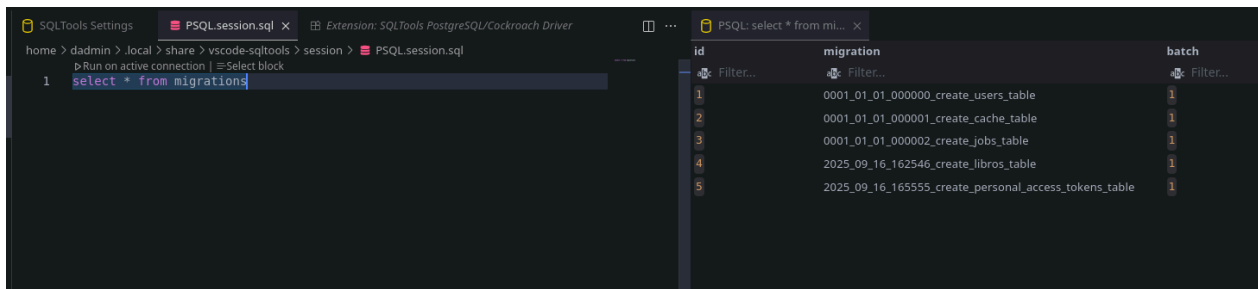
. Instala `SQLTools PostgreSQL/Cockroach Driver`

.

Selecciona o driver de PostgreSQL, e mete os datos que se che proporcionaron para realizar a conexión. Ademais deberás activar o `SSL` e desactivar a opción `rejectUnauthorized`

Conéctate e agora busca a táboa `migrations`

1. e o seu contido. **Entrega captura** do contido de dita táboa.



Explicación En Laravel, cando fas unha **posta en produción**, é moi habitual limpar as cachés. Isto faise porque Laravel garda moita información precompilada en ficheiros de caché para que a aplicación vaia máis rápida.

Se non os limpas, pode pasar que a aplicación quede con **configuracións ou rutas vellas** que non coinciden co novo código que acabas de despregar.

1. Limpa as cachés cos comandos propostos:

```
$ php artisan config:clear      # Se cambiasches algo en `.env` ou nalgún ficheiro de
                                `config/`, tes que rexeneralo.
$ php artisan cache:clear      # Laravel pode gardalas compiladas para mellor rendemento.
$ php artisan route:clear      # As vistas Blade complílanse a PHP e gárdanse en disco.
$ php artisan view:clear       # É a caché xeral que a aplicación pode usar (valores
                                almacenados polo framework ou por ti).
```

Explicación Agora vamos trasladar a nosa aplicación para que Apache poida acceder a ela e servila.

1. Crea un directorio en `/var/www/laravel/`

1..

2. Copia todo o contido do directorio `src` do repositorio descargado a este novo directorio. Utiliza `sudo` para ter permisos de superusuario. Ademais necesitaras copiar arquivos ocultos, así que utiliza o seguinte comando que realiza a copia tanto de ficheiros visibles como ocultos:

```
$ sudo cp -r /home/dadmin/api-rest-en-laravel-t04.03/src/{.,}* /var/www/laravel
```

1. Comproba con `ls -la`

que se copiaron correctamente tódolos ficheiros. **Entrega captura** desta comprobación.

```

dadmin@cvdserver01:/var/www/laravel$ ls -la
total 396
drwxr-xr-x 12 root root 4096 nov 15 15:12 .
drwxr-xr-x  6 root root 4096 nov 15 15:11 ..
drwxr-xr-x  5 root root 4096 nov 15 15:12 app
-rwxr-xr-x  1 root root  425 nov 15 15:12 artisan
drwxr-xr-x  3 root root 4096 nov 15 15:12 bootstrap
-rw-r--r--  1 root root 2468 nov 15 15:12 composer.json
-rw-r--r--  1 root root 313339 nov 15 15:12 composer.lock
drwxr-xr-x  2 root root 4096 nov 15 15:12 config
drwxr-xr-x  5 root root 4096 nov 15 15:12 database
-rw-r--r--  1 root root  258 nov 15 15:12 .editorconfig
-rw-r--r--  1 root root 1200 nov 15 15:12 .env
-rw-r--r--  1 root root  186 nov 15 15:12 .gitattributes
-rw-r--r--  1 root root  283 nov 15 15:12 .gitignore
-rw-r--r--  1 root root  415 nov 15 15:12 package.json
-rw-r--r--  1 root root 1228 nov 15 15:12 phpunit.xml
drwxr-xr-x  2 root root 4096 nov 15 15:12 public
-rw-r--r--  1 root root 3929 nov 15 15:12 README.md
drwxr-xr-x  5 root root 4096 nov 15 15:12 resources
drwxr-xr-x  2 root root 4096 nov 15 15:12 routes
drwxr-xr-x  5 root root 4096 nov 15 15:12 storage
drwxr-xr-x  4 root root 4096 nov 15 15:12 tests
drwxr-xr-x 31 root root 4096 nov 15 15:12 vendor
-rw-r--r--  1 root root  331 nov 15 15:12 vite.config.js

```

Pon ao usuario e grupo `www-data`

1. como propietario de todos estes ficheiros.

Explicación En Laravel o directorio `public/`

contén:

- O ficheiro `index.php`
- : é o punto de entrada da aplicación.
- Os *assets* públicos (CSS, JS, imaxes...).

Todo o resto do proxecto (código, configuración, migracións, etc.) **non debería ser accesible directamente dende o navegador**, por seguridade. Polo tanto na directiva `DocumentRoot`

deberemos especificar este directorio. E na de `Directory`

polo tanto tamén.

Se puxeras como `DocumentRoot`

o cartafol raíz do proxecto, entón o servidor exporía ao público ficheiros sensibles:

- `.env` (contén claves e contrasinais da BD, APIs, etc.)
- Código PHP de `app/` e `routes/`
- Migracións e outros datos internos

Ademais, Laravel segue o patrón **Front Controller**, é dicir, todas as peticións HTTP pasan por un só ficheiro:

`public/index.php`

. Este ficheiro carga o framework e decide que controlador ou ruta executar. Así podes ter rutas limpas tipo

`/usuarios`

sen expoñer a estrutura de ficheiros.

O CSS, JS, imaxes ou fontes da app tamén están en `public/`

. Apache pode servilos directamente, sen pasar polo framework.

1. Realiza unha copia do ficheiro de definición de `virtualhost`

de nome `api.conf`. Modifica os datos para que busque no directorio creado nos pasos anteriores. Esta aplicación resolverá para o nome `api.192-168-56-100.nip.io`. Os *logs* de acceso almacenaranse no ficheiro `/var/log/apache2/api-access.log` e os de erro en `/var/log/apache2/api-error.log`

. **Entrega captura** do ficheiro de definición.

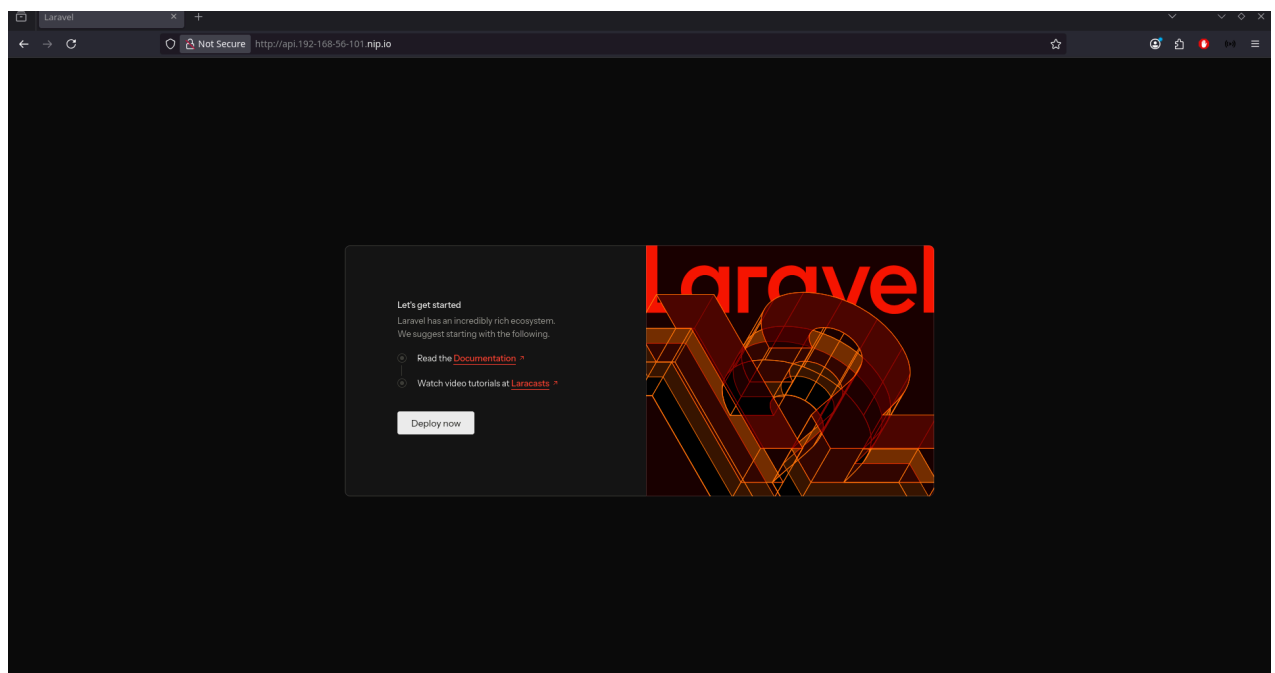
```
GNU nano 7.2                                api.conf
<VirtualHost *:80>
    DocumentRoot "/var/www/laravel/public"
    ServerName api.192-168-56-101.nip.io
    CustomLog ${APACHE_LOG_DIR}/api-access.log
    ErrorLog ${APACHE_LOG_DIR}/api-error.log
</VirtualHost>
<Directory /var/www/laravel/public>
    Options -Indexes +FollowSymLinks
    Require all granted
</Directory>
```

Habilita o sitio.

Recarga a configuración de Apache.

Abre unha ventá privada e pon na URL a dirección `http://api.192-168-56-100.nip.io`

1. . **Entrega capturas** da páxina principal de Laravel.



Explicación Como xa mencionamos anteriormente, en Laravel todas as peticións (agás imaxes, CSS ou JS) deben pasar polo ficheiro `public/index.php`

. Ese ficheiro é o `front controller`

, encargado de cargar o *framework* e resolver a ruta solicitada. Para que isto funcione fáltanos por realizar uns pequenos pasos.

- Activar `mod_rewrite`

permítenos dicirlle a Apache que, se un recurso non existe, redirixa esa petición cara a `index.php`. Se pedimos `https://midominio.com/api/users`

, Apache busca un cartafol chamado `api/` ou un ficheiro chamado `users`. Como non existen, devolvería un **404 Not Found**.

Incluír a directiva `AllowOverride All`

para permitir que Laravel use `.htaccess`. O ficheiro `public/.htaccess` de Laravel xa trae as regras necesarias de reescritura. Con todo, Apache só le ese `.htaccess` se a configuración do servidor lle permite “sobrescribir” opcións a nivel de directorio. Iso contrólase co parámetro `AllowOverride`. Poñendo `AllowOverride All`, autorizamos que Apache aplique as regras de `public/.htaccess` (onde está o `RewriteEngine On` e o `RewriteRule ^ index.php`

-).

Polo tanto vamos a realizar este último paso para que a nosa API-REST quede perfectamente posta en produción.

1. Dende a ventá privada accede a `http://api.192-168-56-100.nip.io/api/libros`

. Deberías recibir un erro `404`.

Activa en Apache o módulo `rewrite`

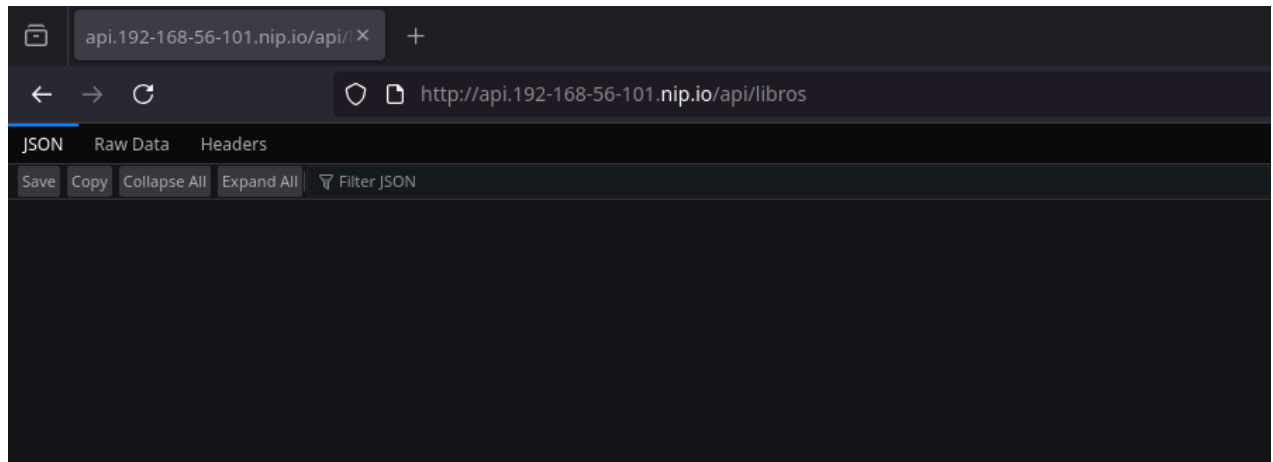
e reinicia Apache.

Engade na definición do `virtualhost`

dentro do bloque `Directory` a directiva `AllowOverride All`. Isto permitirá que se permita utilizar os ficheiros `.htaccess` do directorio `public`

- 1..

2. Agora actualiza a ventá privada e poderás ver xa porque redirecciona correctamente. **Entrega capturas** do que mostra o navegador web.



Explicación Agora vamos a utilizar a API-REST con `cURL` para probar o correcto funcionamento desta. No

`Readme.md`

do proxecto poderás atopar un exemplo de comandos `cURL`.

1. Dende o equipo anfitrión, executa o comando de `cURL` para crear un novo libro. Terás que cambiar o dominio na URL. **Entrega captura** da execución e saída do comando.

```
infe@debian:~/Documents/DAW2Xeral/Despregamento$ curl -i -X POST http://api.192-168-56-101.nip.io/api/libros \
-H "Content-Type: application/json" \
-d '{
  "titulo": "O Señor dos Aneis",
  "autor": "J.R.R. Tolkien",
  "editorial": "Minotauro"
}'
HTTP/1.1 201 Created
Date: Sat, 15 Nov 2025 15:02:06 GMT
Server: Apache/2.4.65 (Debian)
Cache-Control: no-cache, private
Access-Control-Allow-Origin: *
Transfer-Encoding: chunked
Content-Type: application/json

{"titulo":"O Señor dos Aneis","autor":"J.R.R. Tolkien","editorial":"Minotauro","updated_at":"2025-11-15T15:02:06.000000Z","created_at":"2025-11-15T15:02:06.000000Z","id":1}infe@debian:~/Documents/DAW2Xeral/Despregamento$
```

2. Dende o equipo anfitrión, executa o comando de `cURL` que permita ver tódolos libros. **Entrega captura** da execución e saída do comando.

```
infe@debian:~/Documents/DAW2Xeral/Despregamento$ curl -i http://api.192-168-56-101.nip.io/api/libros
HTTP/1.1 200 OK
Date: Sat, 15 Nov 2025 15:03:57 GMT
Server: Apache/2.4.65 (Debian)
Cache-Control: no-cache, private
Access-Control-Allow-Origin: *
Transfer-Encoding: chunked
Content-Type: application/json

[{"id":1,"titulo":"O Señor dos Aneis","autor":"J.R.R. Tolkien","editorial":"Minotauro","created_at":"2025-11-15T15:02:06.000000Z","updated_at":"2025-11-15T15:02:06.000000Z"}]infe@debian:~/Documents/DAW2Xeral/Despregamento$
```

Exercicio 2: Actualización de aplicación Laravel

Explicación Xa temos en produción a nosa aplicación na súa versión 1. Pero agora queremos **actualizar esta na versión 2.0**. Imos ver que pasos seguir para actualizar a nosa aplicación.

1. Copia o `.env` de produción a un lugar seguro. Este ficheiro non é necesario modificalo e necesitarémolo para poñer en produción a versión seguinte. Modifica o usuario propietario por `dadmin`

para obter permisos sobre este. Tamén é boa idea copiar dito ficheiro no anfitrión utilizando `scp`.

Para Apache. Durante o proceso de actualización, a aplicación estará inaccesible. Normalmente avísase aos usuarios da aplicación indicando o día e hora no que se fará dita actualización. Xeralmente díselles ao usuario que a aplicación está en mantemento.

Borra tódolos ficheiros da aplicación, incluídos os ocultos en `/var/www/laravel`

. Comproba co comando `ls -la`

que non queda ningún ficheiro.

Elimina tamén o repositorio clonado. Como temos librerías instaladas poderíamos ter algún conflito.

Clona o repositorio. Sitúate nel e cambia ao `commit`

coa etiqueta `v2.0`.

Instala as dependencias do proxecto con `Composer`

1. .
2. Copia o ficheiro `.env` de produción (o ficheiro que fixemos unha copia) ao repositorio.
3. Realiza a migración de datos para que se actualice o esquema na base de datos de produción.
4. Limpa as *cachés* de Laravel.
5. Copia o código ao seu directorio para que Apache o poida servir e cambia o usuario e grupo propietario a estes.
6. Inicia de novo Apache. Este rematado o proceso de actualización da aplicación.

Explicación A posta en produción da Aplicación xa está realizada. **Non deberemos modificar nada no `virtualhost`**

porque xa estaba configurado para despregar a primeira versión. Agora tan só vamos probala engadindo discos que é a nova funcionalidade que se lle engadiu.

1. Executa o comando de cURL e **entrega captura** da saída.



```
infe@debian:~/Documents/DAW2Xeral/Despregamento$ curl -i -X POST http://api.192-168-56-101.nip.io/api/discos \
-H 'Content-Type: application/json' \
-d '{
  "titulo": "Abbey Road",
  "artista": "The Beatles",
  "genero": "Rock",
  "ano": 1969
}'
HTTP/1.1 201 Created
Date: Sat, 15 Nov 2025 15:24:08 GMT
Server: Apache/2.4.65 (Debian)
Cache-Control: no-cache, private
Access-Control-Allow-Origin: *
Transfer-Encoding: chunked
Content-Type: application/json

{"titulo":"Abbey Road","artista":"The Beatles","genero":"Rock","ano":1969,"updated_at":"2025-11-15T15:24:08.000000Z","created_at":"2025-11-15T15:24:08.000000Z","id":1}infe@debian:~/Documents/DAW2Xeral/Despregamento$
```

```
$ curl -i -X POST http://api.192-168-56-100.nip.io/api/discos \
-H 'Content-Type: application/json' \
-d '{
  "titulo": "Abbey Road",
  "artista": "The Beatles",
  "genero": "Rock",
  "ano": 1969
}'
```

1. Executa o comando cURL e **entrega captura** da saída:

```
infe@debian:~/Documents/DAW2Xeral/Despregamento$ curl -i http://api.192-168-56-101.nip.io/api/discos
HTTP/1.1 200 OK
Date: Sat, 15 Nov 2025 15:25:21 GMT
Server: Apache/2.4.65 (Debian)
Cache-Control: no-cache, private
Access-Control-Allow-Origin: *
Transfer-Encoding: chunked
Content-Type: application/json

[{"id":1,"titulo":"Abbey Road","artista":"The Beatles","genero":"Rock","ano":1969,"created_at":"2025-11-15T15:24:08.000000Z","updated_at":"2025-11-15T15:24:08.000000Z"}]infe@debian:~/Documents/DAW2Xeral/Despregamento$
```

```
$ curl -i http://api.192-168-56-100.nip.io/api/discos
```

Exercicio 3: Despregue de aplicación Vue.js

Explicación Agora vamos a despregar o **frontend** da aplicación. Este para obter o seu contido utilizará a API-REST despregada nos anteriores exercicios. Comezaremos por **obter o código fonte do *frontend***.

1. Realiza un *fork* do seguinte [repositorio](#).
2. Clona o repositorio no teu equipo anfitrión.

Explicación *Frameworks* como **Vue**, **React** ou **Svelte** non funcionan como HTML/JS puro de inicio. O que fas no código fonte (por exemplo, `App.vue`

ou `App.jsx`

) contén:

- **Sintaxe especial:** templates de Vue, JSX en React.
- **Modularidade:** imports/exports de compoñentes, librerías e módulos.
- **Funcionalidades extra:** reactividad, binding, hooks, directivas, etc.

O navegador **non entende directamente esta sintaxe**, só entende HTML, CSS e JavaScript estándar. Por iso necesitamos realizar un **build** do noso código a algo que o navegador poida interpretar.

Durante este proceso:

- **Transpila** a sintaxe especial a JavaScript puro.
- **Agrupa** módulos nun ou varios ficheiros (`bundle`

) para reducir peticións HTTP.

Optimiza: Minimización por exemplo.

Xera ficheiros estáticos: `index.html`

(o punto de entrada), `bundle.js` (todo o código JavaScript compilado), `style.css` (tódolos estilos compilados e minificados) e `assets`

- (imaxes, fontes, iconas, etc.).

Ao final, o servidor só necesita **servir HTML, CSS e JS**. O navegador descarga estes ficheiros e execútalos. Polo tanto podes servir o *frontend* en calquera servidor de ficheiros estáticos (isto xa o realizamos na primeira tarefa).

O proceso de ***build*** vamos a automatizalo co CI/CD en GitLab:

1. Crea un ficheiro de configuración de CI/CD:

- Tan só debe contar unha única etapa de nome `build`.
- Engade como variables o seguinte indicando que a aplicación vaise despregar en produción e engadindo a dirección da API-REST da que se vai consumir. Isto último debes modificalo.

```
variables:
  NODE_ENV: production
  VITE_API_URL: "https://api.exemplo.com" # Modifica isto para poñer a URL da túa API-REST
```

- Esta etapa contará cun único traballo.
 - Este utilizará a imaxe de `node:20`

No traballo deberás:

- Situarte no directorio `src` que contén o código.
- Instalar as dependencias.
- Executar o seguinte comando para actualizar o `.env` coa URL da API-REST: `echo "VITE_API_URL=$VITE_API_URL" > .env`

Realizar a compilación co comando `npm run build`

- . Isto xerará os ficheiros no directorio `dist`,

Para poder descargar o código engade isto na definición do traballo ou nivel de `script`

- ◦ :

```
artifacts:
  paths:
    - dist/ # gardamos os ficheiros xerados
  expire_in: 1 week
```

1. Actualiza o repositorio de GitLab e comproba que todo funciona correctamente. **Entrega o ficheiro** de configuración de CI/CD.

Explicación **Unha vez compilado o código tan só o debemos poñer en produción**. Utilizaremos a mesma máquina virtual que temos con Apache.

1. Descarga o código compilado dende GitLab e descomprímeo.
2. Utiliza o comando `scp` para copiar o directorio `dist` na máquina virtual.
3. Crea un `virtualhost`

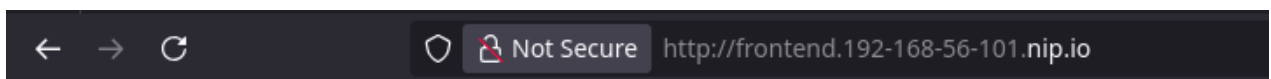
e desprega o código que contén o directorio `dist`. Como URL para servir esta aplicación utiliza `http://frontend.192-168-56-100.nip.io`. **Realiza captura** do `virtualhost`

de definido.

```
GNU nano 7.2                                frontend.conf
VirtualHost *:80>
    DocumentRoot "/var/www/frontend"
    ServerName frontend.192-168-56-101.nip.io
    CustomLog ${APACHE_LOG_DIR}/front-access.log combined
    ErrorLog ${APACHE_LOG_DIR}/front-error.log
    LogLevel warn
</VirtualHost>
<Directory /var/www/frontend>
    Options -Indexes +FollowSymLinks
    Require all granted
    AllowOverride All
</Directory>
```

Nunha ventá privada entra na URL `http://frontend.192-168-56-100.nip.io`

1. para comprobar o correcto funcionamento da aplicación. **Realiza captura** desta aplicación.



Lista de Discos

[+ Engadir disco](#)

NetworkError when attempting to fetch resource.

Exercicio 4: Utilidades

Explicación As `secrets` son **variables sensibles** que non queres gardar directamente no código (como contrasinais, `tokens`, claves API, ou URLs privadas). Serven para que o `pipeline` de CI/CD poida usalas **sen que se filtren** ao repositorio. Estas non se gardan no repositorio, só no servidor CI/CD. Podes usar estas variables como variables de contorno dentro do `pipeline`

.

Vamos utilizar `secrets`

para almacenar a variable de `VITE_API_URL`

que contén a URL da API-REST que consume a aplicación de Vue.js.

1. Vai a `Ajustes → CI/CD → Variables`

no teu proxecto de GitLab que contén a aplicación en Vue.js.

Engade unha nova variable cos seguintes datos:

- Clave: `VITE_API_URL`

Valor: `https://api.exemplo.com`

(modifica pola real)

Marca como **Emmascarada**

- -

Agora modifica o repositorio para eliminar a variable **VITE_API_URL**

do ficheiro de definición de CI/CD.

Sube o repositorio a GitLab. Comproba que o **pipeline**

1. se executa correctamente. **Entrega capturas** do *log* de execución.

```
12 Gitly correlation ID: d443e715fb054c0290792acce7199508
13 Fetching changes with git depth set to 20...
14 Initialized empty Git repository in /builds/a24christianvd/frontend-en-vue.js/.git/
15 Created fresh repository.
16 Checking out 46ce0bad as detached HEAD (ref is main)...
17 Skipping Git submodules setup
18 $ git remote set-url origin "${CI_REPOSITORY_URL}" || echo 'Not a git repository; skipping'
19 Executing "step_script" stage of the job script
20 Using effective pull policy of [always] for container node:20
21 Using docker image sha256:bcd5c4345ce4aaad921c9e4f6773c1554171e357a81825bc81946697756cac for node:20 with digest node@sha256:47dacd49500971c0f6e02323b2d04f6d
f40a933b123889636fc1f76bf69f58a ...
22 $ cd src
23 $ npm install --include=dev
24 added 37 packages, and audited 38 packages in 2s
25 7 packages are looking for funding
26   run 'npm fund' for details
27 found 0 vulnerabilities
28 $ echo "VITE_API_URL=$VITE_API_URL" > .env
29 $ npm run build
30 > src@0.0.0 build
31 > vite build
32 vite v7.1.12 building for production...
33 transforming...
34 ✓ 28 modules transformed.
35 rendering chunks...
36 computing gzip size...
37 dist/index.html          0.38 kB | gzip: 0.26 kB
38 dist/assets/index-06_NCyUN.js 90.59 kB | gzip: 35.48 kB
39 ✓ built in 1.15s
40 Uploading artifacts for successful job
41 Uploading artifacts...
42 src/dist/: found 5 matching artifact files and directories
43 Uploading artifacts as "archive" to coordinator... 201 Created correlation_id=3745bcd6c173424295adb11425bad4e2 id=12100749638 responseStatus=201 Created token=
6b_WTxyWR
44 Cleaning up project directory and file based variables
45 Job succeeded
```

Explicación **DBML** significa **Database Markup Language**. É un **formato de texto plano** creado para describir a estrutura dunha base de datos de forma **humana e legible**, independente do motor de base de datos.

DBML é unha especie de **“linguaxe markdown para bases de datos”**: un formato de texto que describe táboas, columnas e relacións e que se pode transformar en diagramas gráficos ou en SQL real.

Vas obter o DBML da nosa base de datos en PostgreSQL para visualizala nun diagrama.

1. Conéctate a base de datos por medio de **SQLTools**

.

Executa o seguinte **script**

. Para executar na ventá que se abre pegas o código, seleccionalo todo e premes en **F1** e buscas **Run**

Selected Query

- 1..

```
WITH table_columns AS (
  SELECT
    table_name,
    ' ' || column_name || ' ' ||
    CASE
```

```

        WHEN data_type LIKE 'character varying%' THEN 'varchar'
        WHEN data_type LIKE 'timestamp%' THEN 'timestamp'
        ELSE data_type
    END ||
    CASE WHEN column_name = 'id' THEN ' [pk]' ELSE '' END AS col_line
FROM information_schema.columns
WHERE table_schema = 'public'
),
tables_agg AS (
    SELECT
        table_name AS t_name,
        'Table ' || table_name || ' {' || chr(10) ||
        string_agg(col_line, chr(10)) || chr(10) || '}' AS table_block
    FROM table_columns
    GROUP BY table_name
),
fkeys AS (
    SELECT
        tc.table_name AS fk_table,
        kcu.column_name AS fk_column,
        ccu.table_name AS ref_table,
        ccu.column_name AS ref_column
    FROM information_schema.table_constraints AS tc
    JOIN information_schema.key_column_usage AS kcu
        ON tc.constraint_name = kcu.constraint_name
    JOIN information_schema.constraint_column_usage AS ccu
        ON ccu.constraint_name = tc.constraint_name
    WHERE tc.constraint_type = 'FOREIGN KEY'
)
SELECT
    string_agg(table_block, chr(10) || chr(10)) || chr(10) ||
    COALESCE(string_agg('Ref: ' || fk_table || '.' || fk_column || ' > ' || ref_table || '.'
    || ref_column, chr(10)), '') AS dbml_complete
FROM tables_agg
LEFT JOIN fkeys ON TRUE;

```

1. Copia a saída da consulta. Vas a <https://dbdiagram.io/home>

e premes en [Create your diagram](#)

. Pegas a saída da consulta realizada no paso anterior.

Veras un diagrama de táboas da base de datos. **Realiza unha captura** onde se vexa o diagrama que se xerou.